

CONTENTS

Vulnerability Assessment — Student Lab Handout	
PART 1 — What is vulnerability assessment? (read first)	
The four moves — do this for every service	
The deliverable is a RANKED list	
PART 2 — THE RULE (do not skip)	
PART 3 — Your assigned target	
Tier A — local lab target (use this if you can reach the class lab)	
Tier B — public safe target (no lab access needed)	
PART 4 — The tools: YOU fill this in (marked)	
PART 5 — Reading the output (decoder)	
A CVE record — the fields that matter	
searchsploit — reading the table	
nmap --script vuln — reading the output	
nikto — key lines	
False-positive checklist	
Common services → the CVE everyone knows	
PART 6 — WORKSHEET (the main part of your submission)	
A. Recall — your recon output (from the Reconnaissance lab, Journal §2)	
B. searchsploit — is there public exploit code?	
C. NVD / CVE lookup	
D. nmap —script vuln — run: nmap -sV --script vuln -Pn TARGET	

E. nikto — run: nikto -h <WEB-TARGET>

F. TRIAGE TABLE — the point of the whole lab

G. Research questions — answer in your own words, cite the page you used

PART 7 — Setting up the local targets (instructor / advanced students)

One-time: install Podman

DVWA — Damn Vulnerable Web Application → http://<host>:8080

OWASP Juice Shop → http://<host>:3000

Metasploitable 2 → 10.89.1.10 (its own bridge IP, all native ports)

Check everything is up

If a tool isn't installed (Kali has them all; plain Ubuntu/Debian)

PART 8 — Submission

How this is marked

Vulnerability Assessment — Student Lab Handout

Duration: ~35 min in class + finish the report on your own afterwards **Deadline:** submit by _____ today (instructor fills in the time) **What you submit:** ONE document containing (a) the filled-in Part 4 table, (b) the completed Worksheet (Part 6, sections A–G) with your command outputs pasted in as evidence, and (c) your ranked triage table (Part 6-F) **Your target:** you keep the SAME assigned row you had for the Reconnaissance lab — Part 3 explains which target to use **Spot check:** a few students, chosen at random, will be asked to open their submitted document and explain any ranking in it

PART 1 — What is vulnerability assessment? (read first)

Reconnaissance gave you a **map**: open ports, running services, version numbers. Vulnerability assessment is the next phase: **for each service you found, decide whether it has a known weakness, how bad that weakness is, and how urgent it is to fix.**

The order of a pentest (phase 2 is also called the **scanning** phase):

RECON	→	VULN ASSESSMENT	→	EXPLOITATION	→	POST-EXPLOITATION	→	REPORTING
(map)		(rank the weak points)		(break in)		(go deeper)		(write up)

You do NOT log in, run an exploit, or change anything today. That is the next phase. Today stops at *"this door is probably unlocked, here is the evidence, here is where it ranks."*

THE FOUR MOVES — DO THIS FOR EVERY SERVICE

1. MATCH service + version → search NVD / Exploit-DB → CVE number
2. READ the CVE → CVSS score (0-10), is it remote? is it unauthenticated?
3. CHECK searchsploit / Metasploit → is there working public exploit code? (y/n)
4. RANK severity × exposure × public-exploit × asset value → position on the list

A scanner does all four in one second. You learn them by hand so you can tell when the scanner is **wrong**.

THE DELIVERABLE IS A RANKED LIST

Not “I found 9 issues.” A prioritised table your client can act on, most urgent first, each row with a one-line reason.

PART 2 — THE RULE (do not skip)

Vulnerability scanners are LOUDER than recon. They send many more probes and run scripts against the target. A scan aimed at a system you do not own is unauthorised access — a criminal offence (in India, IT Act sections 43 and 66). “I was just running a scanner” is not a defence.

- `searchsploit` and NVD/CVE **lookups** send nothing to any target — always safe.
 - `nmap --script vuln`, `nikto`, `nuclei` **actively probe** — only against the approved targets in Part 3.
 - Your scan should be visible in *your own* logs. If you can’t see it, you’re not in control of it.
-

PART 3 — Your assigned target

You keep the **SAME** row number you used in the **Reconnaissance lab** (roll number, wrapping every 16 — roll 25 → row 9). Your Day 2 Nmap output is the input to today's work.

TIER A — LOCAL LAB TARGET (USE THIS IF YOU CAN REACH THE CLASS LAB)

If the instructor has given you access to the lab network, your active target is the **local vulnerable box**:

What	Address	Use it for
Metasploitable2	10.89.1.10	nmap -sV , nmap --script vuln , searchsploit — a deliberately old Linux box with ~18 services, every version has a real CVE
Metasploitable2's web server	http://10.89.1.10/	the nikto step (it runs Apache 2.2.8 + PHP 5.2)

Everyone shares this one target, so your **raw scan output will look like everyone else's** — that's expected. What must be **your own** is the analysis: the CVE lookups, the false-positive calls, and above all the **triage ranking and its reasons** (Part 6-F). The spot check is on those.

Your instructor may instead point the nikto step at **DVWA** or **OWASP Juice Shop** — these run on a **different address** (the lab host's IP, on ports 8080 and 3000, *not* 10.89.1.10). Use whatever URL the instructor writes on the board.

TIER B — PUBLIC SAFE TARGET (NO LAB ACCESS NEEDED)

If you are working from home with no lab access, use your assigned **active target** from the Reconnaissance lab Part 3 (the vulnweb.com / scanme.nmap.org / ginandjuice.shop / demo.testfire.net host for your row). These are published by their owners (Nmap / Acunetix / PortSwigger / IBM) specifically for scanning practice.

Never point `nmap --script vuln`, `nikto`, or `nuclei` at anything not listed above — no real company, bank, government site, or a friend's server.

PART 4 — The tools: YOU fill this in (marked)

Every tool answers **one question**. If you can say the question, you understand the tool.

How to fill this in: watch the instructor's demo, use the “what you get back” hint, and check Part 5. There is no single correct wording — you are marked on whether the idea is right. Write it yourself.

Tool	The one question it answers (you write)	What you get back (hint)	What you'd do next with the answer (you write)
NVD / CVE search		CVE ID, CVSS score + vector, affected versions, description	
searchsploit		Local Exploit-DB matches: title, path, EDB-ID, type (remote/local/webapps/dos)	
nmap <code>--script vuln</code>		Per-port: CVE IDs, “VULNERABLE”/“LIKELY VULNERABLE”, CVSS, references	
nikto		Web server + version, outdated software, dangerous HTTP methods, interesting files, missing headers	
nuclei (optional)		Template-matched findings tagged by severity (info/low/medium/high/critical)	

Hint for “the question”: start with “Does this exact version...”, “Is there public code...”, “What weaknesses does this web server...”. Hint for “next”: look up the CVSS, read the CVE conditions, mark it confirmed / needs-check, add it to the triage table, hand it to Phase 4.

PART 5 — Reading the output (decoder)

Every line of output is a fact about the target. This is the part most people skip.

A CVE RECORD — THE FIELDS THAT MATTER

Field	What it tells you
CVE ID	<code>CVE-2011-2523</code> — the unique name. Search this to find everything else.
CVSS base score	<code>0.0</code> — <code>10.0</code> . Bands: 0.1–3.9 Low · 4.0–6.9 Medium · 7.0–8.9 High · 9.0–10.0 Critical
Attack Vector (AV)	<code>Network</code> = exploitable remotely (worst) · <code>Adjacent</code> = same LAN · <code>Local</code> = needs a shell already · <code>Physical</code>
Privileges Required (PR)	<code>None</code> = no login needed (worse) · <code>Low</code> / <code>High</code> = needs an account first
User Interaction (UI)	<code>None</code> = fully automatic · <code>Required</code> = needs a victim to click something
Affected versions	The exact version range. Your banner must fall inside it — or it's a false positive.

The two questions that decide urgency: is it `AV:Network` and is it `PR:None`? A remote, unauthenticated bug is the one that ruins your week.

SEARCHSPLOIT — READING THE TABLE

Exploit Title	Path
-----	-----

vsftpd 2.3.4 - Backdoor Command Execution	unix/remote/49757.py
vsftpd 2.3.4 - Backdoor Command Execution (Metasploit)	unix/remote/17491.rb

Column	Meaning
Title	The product + version + what the exploit does. Match the version to <i>your</i> banner exactly.
Path	File under <code>/usr/share/exploitdb/exploits/...</code> . The number is the EDB-ID .
<code>(Metasploit)</code>	A ready-to-run module exists — this finding jumps up the ranking.
<code>remote/</code>	Category: <code>remote</code> (over network — worst), <code>local</code> (needs access already), <code>webapps</code> , <code>dos</code>

`searchsploit -x <path>` shows the exploit; `searchsploit -m <EDB-ID>` copies it to your folder. **Finding an exploit is not permission to run it — that is Phase 4.**

NMAP `--SCRIPT VULN` — READING THE OUTPUT

```

PORT      STATE SERVICE
21/tcp    open  ftp
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|   vsftpd version 2.3.4 backdoor
|     State: VULNERABLE (Exploitable)
|     IDs:  CVE:CVE-2011-2523

```

You see	It means
<code>VULNERABLE (Exploitable)</code>	The script actually triggered/confirmed the condition — high confidence
<code>LIKELY VULNERABLE / VULNERABLE (DoS)</code>	Version-based guess — validate before you trust it
<code>Couldn't find ... / no output</code>	Script ran, found nothing (not proof it's safe — just that this check missed)

NIKTO — KEY LINES

Line	What it tells you
Server: Apache/2.2.8	Web server + version — search it for CVEs
Retrieved x-powered-by header: PHP/5.2.4	Backend language + version — often the softer target
OSVDB-xxxx / CVE-xxxx entries	Specific known issues nikto matched
Allowed HTTP Methods: ... PUT DELETE	Dangerous methods enabled — possible file upload
/admin/, /phpinfo.php, /test/ found	Interesting paths to look at (in a browser, not with an exploit)
The anti-clickjacking X-Frame-Options header is not present	Missing hardening — low severity, still a finding

FALSE-POSITIVE CHECKLIST

Before a scanner line goes in your report as a **finding**, check:

- ☐ Does my banner version fall inside the CVE's *affected versions*?
- ☐ Could this be a **backported patch** (distro fixed it, version string unchanged)?
- ☐ Does the CVE need a **config or module** that may not be enabled here?
- ☐ Did the script say `VULNERABLE (Exploitable)` (confirmed) or just a version guess?

Mark every finding: **CONFIRMED**, **NEEDS-CHECK**, or **FALSE POSITIVE**.

COMMON SERVICES → THE CVE EVERYONE KNOWS

Service / version	Known issue	Auth needed?
vsftpd 2.3.4	Backdoor — smiley in the username opens a root shell (CVE-2011-2523)	No
UnrealIRCd 3.2.8.1	Backdoored release — command execution (CVE-2010-2075)	No
Samba 3.0.20–3.0.25	usermap_script — command injection (CVE-2007-2447)	No
distccd v1	Distributed compiler accepts arbitrary commands (CVE-2004-2687)	No
Apache 2.2.8 (2008)	Many CVEs — most need a specific module or config to be exploitable	Varies
OpenSSH 4.7p1	Username enumeration (CVE-2018-15473); no reliable unauthenticated RCE	No (for enumeration)
MySQL / Postgres open to the network	Weak or blank credentials — not a CVE, a misconfiguration	—

PART 6 — WORKSHEET (the main part of your submission)

Name: _____ Roll no: _____ Date: _____ Row #
 (same as recon lab): _____ My target this lab: ☐ Tier A 10.89.1.10 ☐ Tier B
 _____ Authorization: my target is on the Part 3 approved list.

Below, TARGET = your active target from Part 3.

A. RECALL — YOUR RECON OUTPUT (FROM THE RECONNAISSANCE LAB, JOURNAL §2)

1. Paste the open ports + service + version table you produced in the recon lab. If you don't have it, re-run `nmap -sV -Pn TARGET` and paste it now.

Port	Service	Version

B. SEARCHSPLOIT — IS THERE PUBLIC EXPLOIT CODE?

Run `searchsploit <product> <version>` for **each** service that shows a version number.

2. Fill one row per service:

Service + version	searchsploit result (title, or “none”)	EDB- ID	Metasploit module? (y/n)

3. Which service has the most dangerous public exploit, and why? (1 sentence)

C. NVD / CVE LOOKUP

Pick **two** services from your Part A port list — ideally ones that showed a version number. For each, search `https://nvd.nist.gov` (or the web) for the product name + version + `CVE`.

4. Service 1: _____ CVE: _____ CVSS score + band: _____
 _____ Attack Vector: _____ Privileges Required: _____ → in one line, what does it let an attacker do?

5. Service 2: _____ CVE: _____ CVSS score + band: _____
 _____ Attack Vector: _____ Privileges Required: _____ → in one line, what does it let an attacker do?

6. Of these two, which is more urgent and why? (mention remote/unauthenticated)

D. NMAP —SCRIPT VULN — RUN: `NMAP -SV --SCRIPT VULN -PN TARGET`

(Takes 3–8 minutes. Let it finish.)

7. List every CVE / issue the scripts flagged, and how confident the script was:

Port	Issue / CVE	Script said (Exploitable / Likely / DoS)	CONFIRMED / NEEDS-CHECK / FALSE POSITIVE — why

8. Pick one “**LIKELY VULNERABLE**” line. How would you check whether it’s real? (2 sentences)

E. NIKTO — RUN: `NIKTO -H <WEB-TARGET>`

Tier A: `nikto -h http://10.89.1.10/` — unless the instructor gave you a DVWA / Juice Shop URL, in which case use that. Tier B: `nikto -h http://TARGET` (your assigned host). If the site is HTTPS only, use `https://`.

9. Web server software + version: _____
10. Backend language / framework + version (if shown): _____
11. Two specific issues nikto reported (dangerous method, outdated component, exposed path, missing header): _____

12. Is the web server version outdated? Search `<web server> latest version`, compare, and say “current” or “outdated (latest is _____)”: _____

F. TRIAGE TABLE — THE POINT OF THE WHOLE LAB

Combine everything above. One row per real finding. Rank them, most urgent first.

Rank	Finding (service + issue)	CVSS band	Exposed?	Public exploit?	Asset value	One-line reason for this rank
1						
2						
3						
4						
5						

13. **Justify your #1 in 2 sentences** — why does it beat everything below it?

14. **Did a lower-severity finding outrank a higher-severity one?** If yes, which and why? If no, explain why severity alone was enough this time.

G. RESEARCH QUESTIONS — ANSWER IN YOUR OWN WORDS, CITE THE PAGE YOU USED

15. **CVSS:** what score range is “Critical”? What do `AV:Network` and `PR:None` each mean, and why does each one make a vulnerability worse?

16. **Vuln assessment vs penetration test:** you confirmed a weakness today but did not use it. In 3 sentences, what would the *exploitation* phase add, and why keep the phases separate?

17. **False positives:** describe two concrete reasons a scanner reports a vulnerability that isn’t actually exploitable on that host.

18. **One CVE deep-dive:** take one CVE you found (or `CVE-2011-2523` for vsftpd 2.3.4).

- In one sentence, what does it let an attacker do?
- Authenticated or unauthenticated?
- Roughly when was it published, and is a patch available?

19. **Vulnerability management:** a company runs a scanner once a year. Give two reasons that is not enough, and name one thing an *ongoing* program does instead (scan cadence, patch SLA, asset inventory...).

20. **Responsible disclosure:** you find a serious unpatched flaw in software made by a company that did **not** hire you. What is the accepted process — and what should you *not* do?
 21. **The law:** find your country's law on unauthorised access to a computer system. Write the act name, the section number, and the maximum penalty. (You wrote this for the recon lab — confirm it's the same, or correct it.)
 22. **Paste all your raw command outputs** below as evidence (or attach screenshots).
-

PART 7 — Setting up the local targets (instructor / advanced students)

The Tier A targets run as **rootless/rootful Podman containers inside a WSL2 Ubuntu distro**. Kali (the attacker box) reaches them because all WSL distros share one network namespace.

ONE-TIME: INSTALL PODMAN

```
# in the Ubuntu WSL distro
sudo apt update
sudo apt install -y podman passt slirp4netns fuse-overlayfs uidmap crun

# WSL kernel has no iptables-NAT – tell Podman's network layer not to need it
sudo mkdir -p /etc/containers
printf '[network]\nfirewall_driver = "none"\n' | sudo tee
/etc/containers/containers.conf

# keep containers alive across reboots
sudo loginctl enable-linger "$USER"
systemctl --user enable podman-restart.service
sudo systemctl enable podman-restart.service
```

DVWA — DAMN VULNERABLE WEB APPLICATION →**HTTP://<HOST>:8080**

```
podman run -d --name dvwa --restart always -p 8080:80 \
  docker.io/vulnerables/web-dvwa:latest
```

First use: browse **:8080**, log in **admin** / **password**, click **Create / Reset Database**.

OWASP JUICE SHOP → HTTP://<HOST>:3000

```
podman run -d --name juiceshop --restart always -p 3000:3000 \
  docker.io/bkimminich/juice-shop:latest
```

METASPLOITABLE 2 → 10.89.1.10 (ITS OWN BRIDGE IP, ALL NATIVE PORTS)

```
# dedicated bridge with a KNOWN subnet, so the --ip below is valid and
predictable
sudo podman network create --subnet 10.89.1.0/24 labnet

# the image's default command (services.sh && bash) exits when run detached, so
wrap it
sudo podman run -d --name msf2 --restart always \
  --network labnet --ip 10.89.1.10 --hostname metasploitable \
  docker.io/tleemcjr/metasploitable2 \
  sh -c '/bin/services.sh; exec tail -f /dev/null'

# confirm it took the address and is not restart-looping (RestartCount should
stay 0)
sudo podman inspect msf2 \
  --format 'IP={{.NetworkSettings.Networks.labnet.IPAddress}} restarts=
{{.RestartCount}}'
```

CHECK EVERYTHING IS UP

```
podman ps                # dvwa, juiceshop
sudo podman ps           # msf2
curl -I http://localhost:8080/ ; curl -I http://localhost:3000/
sudo podman exec -it msf2 bash # shell inside Metasploitable2

# from Kali (find the shared WSL IP with: ip -4 addr show eth0 ):
nmap -sV 10.89.1.10
nmap -sV <shared-wsl-ip> -p 8080,3000
```

Reachability: all WSL distros (Ubuntu, Kali) share one `eth0` and one IP, so from Kali the DVWA / Juice Shop ports are on that shared IP (`:8080` / `:3000`) — and on the Windows host at `localhost:8080` / `:3000` . Metasploitable2 has its own bridge address `10.89.1.10` , reachable from Kali and the Ubuntu host (not from Windows without an added route — and the attacker box is Kali, so that's fine).

Keep the lab isolated. Metasploitable2 has no defences — never port-forward it to a real network or the internet.

IF A TOOL ISN'T INSTALLED (KALI HAS THEM ALL; PLAIN UBUNTU/DEBIAN)

```
sudo apt install -y nmap nikto exploitdb # exploitdb provides `searchsploit`
# nuclei (optional): download the release binary from the ProjectDiscovery
GitHub
```

PART 8 — Submission

Put everything in **one document** and submit by the deadline on page 1:

- The **Part 4 table**, both blank columns filled in your own words.
- Worksheet sections **A–E**: every answer, plus raw command outputs pasted into question 22.
- Worksheet section **F**: the ranked triage table + your #1 justification.

- Worksheet section **G** (questions 15–21): answered in your own words, each with the link or page you used.

Be ready to open the document and **explain any ranking** if you are picked for the spot check.

HOW THIS IS MARKED

What	Which questions	Checks	Weight
Part 4 table filled in	Part 4	You understand what each tool is <i>for</i>	10%
Commands actually run	outputs in Q22	You ran the tools on your assigned target	15%
Version → CVE lookup	4, 5, 18	You can turn a version string into a specific known vulnerability	15%
Reading CVSS + exposure	6, 15	You can read severity and judge urgency	15%
False-positive judgement	7, 8, 17	You treat scanner output as leads, not findings	15%
Triage table + justification	F, 13, 14	You can turn raw findings into a prioritised, defensible list	20%
Research questions	16, 19, 20, 21	You looked things up and understood them	10%

Answers copied from another student, with no understanding shown in the spot check, score zero.

The most important thing to take away: a scanner gives you 30 leads; the job is turning them into the 3 things the client should fix first — and being able to say *why those three*.